

Lecture 09 - Texturing

This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

Source chunk	course_text_parts/03_lectures/09_texturing.txt
Extracted slide pages	133
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Textures: Sampled Data for Shading
Source cue	■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data

Professor-style explanation	The slide 'Textures: Sampled Data for Shading' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Textures: Sampled Data for Shading. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Textures: Sampled Data for Shading'?

Page 3	Texturing Example
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Texturing Example' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Texturing Example. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Texturing Example' into a causal sentence instead of repeating the slide title?

Page 4	Texture Mapping
--------	-----------------

Source cue	■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader
Professor-style explanation	The slide 'Texture Mapping' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texture Mapping. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texture Mapping'?

Page 5	Why Textures Matter
Source cue	■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient
Professor-style explanation	The slide 'Why Textures Matter' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.

Technical commentary	This slide is about Why Textures Matter. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Why Textures Matter'?

Page 6	Textures Are More Than Color Images
Source cue	■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo
Professor-style explanation	The slide 'Textures Are More Than Color Images' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Textures Are More Than Color Images. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Textures Are More Than Color Images'?

Page 7	Texturing Demo
Source cue	Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness

Professor-style explanation	The slide 'Texturing Demo' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Texturing Demo. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Texturing Demo'?
Page 8	Overview
Source cue	9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping
Professor-style explanation	The slide 'Overview' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: 9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Overview. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: 9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.

Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Overview'?
Page 9	9.1 Texture Objects, Texels, Formats, and Color Space
Source cue	What texture data stores and how the GPU interprets it
Professor-style explanation	The slide '9.1 Texture Objects, Texels, Formats, and Color Space' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: What texture data stores and how the GPU interprets it. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about 9.1 Texture Objects, Texels, Formats, and Color Space. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: What texture data stores and how the GPU interprets it
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '9.1 Texture Objects, Texels, Formats, and Color Space'?
Page 10	From Image File to GPU Texture Object
Source cue	■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline
Professor-style explanation	The slide 'From Image File to GPU Texture Object' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about From Image File to GPU Texture Object. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'From Image File to GPU Texture Object'?

Page 11	Texels, Channels, and Meaning
Source cue	■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA
Professor-style explanation	The slide 'Texels, Channels, and Meaning' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Texels, Channels, and Meaning. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Texels, Channels, and Meaning'?

Page 12	Texture Targets and Dimensionality
Source cue	■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions

Professor-style explanation	The slide 'Texture Targets and Dimensionality' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Targets and Dimensionality. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Targets and Dimensionality'?

Page 13	Internal Texture Formats
Source cue	■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels
Professor-style explanation	The slide 'Internal Texture Formats' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Internal Texture Formats. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels

Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Internal Texture Formats'?
Page 14	Numeric Texture Representations
Source cue	■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0
Professor-style explanation	The slide 'Numeric Texture Representations' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Numeric Texture Representations. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Numeric Texture Representations'?
Page 15	Color Textures vs. Data Textures
Source cue	■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color
Professor-style explanation	The slide 'Color Textures vs. Data Textures' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Color Textures vs. Data Textures. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Color Textures vs. Data Textures'?

Page 16	2D Texture Access
Source cue	■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated
Professor-style explanation	The slide '2D Texture Access' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 2D Texture Access. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '2D Texture Access'?

Page 17	Bilinear Interpolation
Source cue	■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations (Notation: i_j) / $s-s / f = (1 - \alpha) \cdot f + \alpha \cdot f_\alpha =$

Professor-style explanation	The slide 'Bilinear Interpolation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations (Notation: $ij\ ij\ ij$) / $s-s / f = (1 - \alpha) \cdot f + \alpha \cdot f \alpha =$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Bilinear Interpolation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations (Notation: $ij\ ij\ ij$) / $s-s / f = (1 - \alpha) \cdot f + \alpha \cdot f \alpha =$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Bilinear Interpolation' into a causal sentence instead of repeating the slide title?
Page 18	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 19	Texture Memory Footprint
Source cue	■ Texture memory depends on resolution and format / $\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}$ / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB

Professor-style explanation	The slide 'Texture Memory Footprint' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Texture memory depends on resolution and format / $\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}$ / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Texture Memory Footprint. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Texture memory depends on resolution and format / $\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}$ / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Texture Memory Footprint'?

Page 20	Texture Compression and Asset Pipelines
Source cue	■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats
Professor-style explanation	The slide 'Texture Compression and Asset Pipelines' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Texture Compression and Asset Pipelines. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats

Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Texture Compression and Asset Pipelines'?
Page 21	Steps Involved in Texturing
Source cue	■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value
Professor-style explanation	The slide 'Steps Involved in Texturing' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Steps Involved in Texturing. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Steps Involved in Texturing'?
Page 22	9.2 Texture Coordinates, UV Mapping, and Wrapping
Source cue	Aligning texture space with geometry

Professor-style explanation	The slide '9.2 Texture Coordinates, UV Mapping, and Wrapping' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Aligning texture space with geometry. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 9.2 Texture Coordinates, UV Mapping, and Wrapping. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Aligning texture space with geometry
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '9.2 Texture Coordinates, UV Mapping, and Wrapping'?

Page 23	Texture Coordinate System 1/2
Source cue	■ Object space: 3D vertices in model coordinate system / (x, y, z) / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / (s, t, r, q)
Professor-style explanation	The slide 'Texture Coordinate System 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Object space: 3D vertices in model coordinate system / (x, y, z) / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / (s, t, r, q). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Coordinate System 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Object space: 3D vertices in model coordinate system / (x, y, z) / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / (s, t, r, q)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate System 1/2'?

Page 24	Texture Coordinate System 2/2
Source cue	$[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / s, t, r, q $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by with / (s)
Professor-style explanation	The slide 'Texture Coordinate System 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / s, t, r, q $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by with / (s). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Coordinate System 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / s, t, r, q $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by with / (s)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate System 2/2'?
Page 25	Texture Coordinate Specification
Source cue	■ 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex g of a polygon, texture coordinates t are specified / • Texture coordinates t are obtained from parametrization algorithms / • Texture coordinates t are linearly interpolated over a polygon (analogous to Gouraud shading)

Professor-style explanation	The slide 'Texture Coordinate Specification' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex g of a polygon, texture coordinates t are specified / • Texture coordinates t are obtained from parametrization algorithms / • Texture coordinates t are linearly interpolated over a polygon (analogous to Gouraud shading). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Coordinate Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex g of a polygon, texture coordinates t are specified / • Texture coordinates t are obtained from parametrization algorithms / • Texture coordinates t are linearly interpolated over a polygon (analogous to Gouraud shading)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate Specification'?
Page 26	Disc Parametrization
Source cue	r xy / ■ Circular disc with radius r, centered in the plane / $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ / • g with and / $p = (s, t) \in [0, 1]^2$ $s = \theta$ $t = a$
Professor-style explanation	The slide 'Disc Parametrization' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: r xy / ■ Circular disc with radius r, centered in the plane / $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ / • g with and / $p = (s, t) \in [0, 1]^2$ $s = \theta$ $t = a$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Disc Parametrization. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: r xy / ■ Circular disc with radius r, centered in the plane / $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ / • g with and / $p = (s, t) \in [0, 1]^2$ $s = \theta$ $t = a$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Disc Parametrization' into a causal sentence instead of repeating the slide title?
Page 27	Cylinder Parametrization
Source cue	xy r = 1 h z / ■ Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) / p = (x, y, z) = (r · cos(θ), r · sin(θ), z) θ ∈ [0, 2π] z ∈ [0, h] / • g with and
Professor-style explanation	The slide 'Cylinder Parametrization' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: xy r = 1 h z / ■ Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) / p = (x, y, z) = (r · cos(θ), r · sin(θ), z) θ ∈ [0, 2π] z ∈ [0, h] / • g with and. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Cylinder Parametrization. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: xy r = 1 h z / ■ Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) / p = (x, y, z) = (r · cos(θ), r · sin(θ), z) θ ∈ [0, 2π] z ∈ [0, h] / • g with and
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Cylinder Parametrization' into a causal sentence instead of repeating the slide title?
Page 28	Two-Part Mapping 1/2
Source cue	■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / p = (x, y, z) p = (s, t) p
Professor-style explanation	The slide 'Two-Part Mapping 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / p = (x, y, z) p = (s, t) p. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Two-Part Mapping 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / $p = (x, y, z)$ $p = (s, t)$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Two-Part Mapping 1/2'?

Page 29	Two-Part Mapping 2/2
Source cue	■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured
Professor-style explanation	The slide 'Two-Part Mapping 2/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Two-Part Mapping 2/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Two-Part Mapping 2/2'?

Page 30	Two-Part Mapping Example
Source cue	■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$

Professor-style explanation	The slide 'Two-Part Mapping Example' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: ■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Two-Part Mapping Example. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: ■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Two-Part Mapping Example' changes positions before rasterization?

Page 31	Projection Calculation Variations
Source cue	O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O
Professor-style explanation	The slide 'Projection Calculation Variations' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Calculation Variations. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.

Check yourself	Can you explain how 'Projection Calculation Variations' changes positions before rasterization?
Page 32	Texture Coordinate Transformation
Source cue	■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates
Professor-style explanation	The slide 'Texture Coordinate Transformation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Coordinate Transformation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate Transformation'?
Page 33	Demo - Texture Coordinate Transformation
Source cue	Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0
Professor-style explanation	The slide 'Demo - Texture Coordinate Transformation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Demo - Texture Coordinate Transformation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Demo - Texture Coordinate Transformation'?

Page 34	Height Field Texturing Example
Source cue	■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r
Professor-style explanation	The slide 'Height Field Texturing Example' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Height Field Texturing Example. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Height Field Texturing Example'?

Page 35	9.3 Sampling, Filtering, Mipmaps, and Anisotropy
Source cue	Reconstructing texture values under magnification and minification

Professor-style explanation	The slide '9.3 Sampling, Filtering, Mipmaps, and Anisotropy' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Reconstructing texture values under magnification and minification. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about 9.3 Sampling, Filtering, Mipmaps, and Anisotropy. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Reconstructing texture values under magnification and minification
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '9.3 Sampling, Filtering, Mipmaps, and Anisotropy'?

Page 36	Forward vs. Inverse Mapping
Source cue	■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture
Professor-style explanation	The slide 'Forward vs. Inverse Mapping' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Forward vs. Inverse Mapping. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture

Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Forward vs. Inverse Mapping'?
Page 37	Forward Mapping
Source cue	■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory
Professor-style explanation	The slide 'Forward Mapping' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Forward Mapping. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Forward Mapping'?
Page 38	Inverse Mapping 1/2
Source cue	■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'

Professor-style explanation	The slide 'Inverse Mapping 1/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Inverse Mapping 1/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Inverse Mapping 1/2'?

Page 39	Inverse Mapping 2/2
Source cue	■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram
Professor-style explanation	The slide 'Inverse Mapping 2/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Inverse Mapping 2/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram

Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Inverse Mapping 2/2'?
Page 40	Magnification vs. Minification
Source cue	■ Magnification / • Map few texels onto many pixels / • Two filtering approaches / • Nearest: Take the nearest texel / • Bilinear interpolation: Interpolation between 4 nearest texels
Professor-style explanation	The slide 'Magnification vs. Minification' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Magnification / • Map few texels onto many pixels / • Two filtering approaches / • Nearest: Take the nearest texel / • Bilinear interpolation: Interpolation between 4 nearest texels. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Magnification vs. Minification. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Magnification / • Map few texels onto many pixels / • Two filtering approaches / • Nearest: Take the nearest texel / • Bilinear interpolation: Interpolation between 4 nearest texels
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Magnification vs. Minification'?
Page 41	Nearest vs. Linear Interpolation
Source cue	■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y

Professor-style explanation	The slide 'Nearest vs. Linear Interpolation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Nearest vs. Linear Interpolation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Nearest vs. Linear Interpolation'?

Page 42	Minification Filtering Requirements
Source cue	■ Preimage integration with insufficient filtering results in aliasing / artifacts
Professor-style explanation	The slide 'Minification Filtering Requirements' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Preimage integration with insufficient filtering results in aliasing / artifacts. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Minification Filtering Requirements. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Preimage integration with insufficient filtering results in aliasing / artifacts
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Minification Filtering Requirements'?

Page 43	Demo - Magnification Filtering: Nearest vs Linear
Source cue	Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid
Professor-style explanation	The slide 'Demo - Magnification Filtering: Nearest vs Linear' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Demo - Magnification Filtering: Nearest vs Linear. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Magnification Filtering: Nearest vs Linear'?
Page 44	MIP Mapping (multum in parvo = 'much in little')
Source cue	■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer
Professor-style explanation	The slide 'MIP Mapping (multum in parvo = 'much in little')' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.

Technical commentary	This slide is about MIP Mapping (multum in parvo = 'much in little'). The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'MIP Mapping (multum in parvo = 'much in little')'?

Page 45	MIP Map Pyramid 1/2
Source cue	■ MIP Map-Level 0 : Texture in original size / $2N-1$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N/20$
Professor-style explanation	The slide 'MIP Map Pyramid 1/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ MIP Map-Level 0 : Texture in original size / $2N-1$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N/20$. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about MIP Map Pyramid 1/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ MIP Map-Level 0 : Texture in original size / $2N-1$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N/20$
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'MIP Map Pyramid 1/2'?

Page 46	MIP Map Pyramid 2/2
Source cue	■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage

Professor-style explanation	The slide 'MIP Map Pyramid 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about MIP Map Pyramid 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'MIP Map Pyramid 2/2' into a causal sentence instead of repeating the slide title?

Page 47	MIP Mapping Memory
Source cue	■ 33% more memory is needed
Professor-style explanation	The slide 'MIP Mapping Memory' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ 33% more memory is needed. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about MIP Mapping Memory. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ 33% more memory is needed
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'MIP Mapping Memory' into a causal sentence instead of repeating the slide title?

Page 48	MIP Mapping Procedure
Source cue	$\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to

Professor-style explanation	The slide 'MIP Mapping Procedure' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: $\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about MIP Mapping Procedure. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: $\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'MIP Mapping Procedure' into a causal sentence instead of repeating the slide title?
Page 49	LoD Calculation 1/2
Source cue	■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2N$ / • 2D texture has texels with (power-of-two texture)
Professor-style explanation	The slide 'LoD Calculation 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2N$ / • 2D texture has texels with (power-of-two texture). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about LoD Calculation 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2N$ / • 2D texture has texels with (power-of-two texture)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'LoD Calculation 1/2'?

Page 50	LoD Calculation 2/2
Source cue	■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A$ / • Length of square's side is
Professor-style explanation	The slide 'LoD Calculation 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A$ / • Length of square's side is. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about LoD Calculation 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A$ / • Length of square's side is
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'LoD Calculation 2/2' into a causal sentence instead of repeating the slide title?
Page 51	Demo - Minification, Mipmaps, and Mipmap Level Visualization
Source cue	Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels
Professor-style explanation	The slide 'Demo - Minification, Mipmaps, and Mipmap Level Visualization' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Demo - Minification, Mipmaps, and Mipmap Level Visualization. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.

Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Minification, Mipmaps, and Mipmap Level Visualization'?
Page 52	Anisotropic Filtering
Source cue	■ Approximation of preimage through several squares
Professor-style explanation	The slide 'Anisotropic Filtering' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Approximation of preimage through several squares. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Anisotropic Filtering. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Approximation of preimage through several squares
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Anisotropic Filtering'?
Page 53	Demo - Anisotropic Filtering
Source cue	Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10
Professor-style explanation	The slide 'Demo - Anisotropic Filtering' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Demo - Anisotropic Filtering. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10

Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Demo - Anisotropic Filtering' changes positions before rasterization?
Page 54	9.4 Modern OpenGL Texture Pipeline
Source cue	Texture objects, sampler uniforms, and shader-based access
Professor-style explanation	The slide '9.4 Modern OpenGL Texture Pipeline' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Texture objects, sampler uniforms, and shader-based access. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 9.4 Modern OpenGL Texture Pipeline. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Texture objects, sampler uniforms, and shader-based access
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '9.4 Modern OpenGL Texture Pipeline'?
Page 55	OpenGL Texturing
Source cue	■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture
Professor-style explanation	The slide 'OpenGL Texturing' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about OpenGL Texturing. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Texturing'?

Page 56	OpenGL Minification & Magnification 1/3
Source cue	■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation
Professor-style explanation	The slide 'OpenGL Minification & Magnification 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Minification & Magnification 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 1/3'?

Page 57	OpenGL Minification & Magnification 2/3
Source cue	■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...

Professor-style explanation	The slide 'OpenGL Minification & Magnification 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Minification & Magnification 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 2/3'?

Page 58	OpenGL Minification & Magnification 3/3
Source cue	■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...
Professor-style explanation	The slide 'OpenGL Minification & Magnification 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Minification & Magnification 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 3/3'?
Page 59	Demo - Texture Units and Sampler Uniforms
Source cue	Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0
Professor-style explanation	The slide 'Demo - Texture Units and Sampler Uniforms' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Demo - Texture Units and Sampler Uniforms. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Texture Units and Sampler Uniforms'?
Page 60	Texture Wrapping 1/4
Source cue	■ Texture wrapping specifies border handling behavior when accessing texture outside the / [0, 1]D / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension

Professor-style explanation	The slide 'Texture Wrapping 1/4' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Texture wrapping specifies border handling behavior when accessing texture outside the $[0, 1]^D$ / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Texture Wrapping 1/4. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Texture wrapping specifies border handling behavior when accessing texture outside the $[0, 1]^D$ / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Texture Wrapping 1/4'?

Page 61	Texture Wrapping 2/4
Source cue	■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER
Professor-style explanation	The slide 'Texture Wrapping 2/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Texture Wrapping 2/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Wrapping 2/4'?
Page 62	Texture Wrapping 3/4
Source cue	■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■ GL_MIRRORED_REPEAT
Professor-style explanation	The slide 'Texture Wrapping 3/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■ GL_MIRRORED_REPEAT. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Wrapping 3/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■ GL_MIRRORED_REPEAT
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Wrapping 3/4'?
Page 63	Texture Wrapping 4/4
Source cue	■ OpenGL code snippets / 1 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); / 2 glTexParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT); / 1 const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f }; / 3 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER);

Professor-style explanation	The slide 'Texture Wrapping 4/4' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ OpenGL code snippets / 1 <pre>glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); / 2 glTextureParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT); / 1 const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f }; / 3 glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER).</pre> The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texture Wrapping 4/4. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ <pre>OpenGL code snippets / 1 glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); / 2 glTextureParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT); / 1 const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f }; / 3 glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER);</pre>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texture Wrapping 4/4'?
Page 64	Texel Value Usage
Source cue	■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C

Professor-style explanation	The slide 'Texel Value Usage' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texel Value Usage. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texel Value Usage'?
Page 65	Texturing in GLSL Shaders 1/3
Source cue	■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p);
Professor-style explanation	The slide 'Texturing in GLSL Shaders 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Texturing in GLSL Shaders 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p);
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 1/3'?
Page 66	Texturing in GLSL Shaders 2/3
Source cue	■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix;
Professor-style explanation	The slide 'Texturing in GLSL Shaders 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texturing in GLSL Shaders 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix;
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 2/3'?
Page 67	Texturing in GLSL Shaders 3/3
Source cue	■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_;

Professor-style explanation	The slide 'Texturing in GLSL Shaders 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texturing in GLSL Shaders 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_;
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 3/3'?

Page 68	9.5 Normal Mapping
Source cue	Encoding surface detail in texture space
Professor-style explanation	The slide '9.5 Normal Mapping' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Encoding surface detail in texture space. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 9.5 Normal Mapping. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Encoding surface detail in texture space
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '9.5 Normal Mapping'?

Page 69	Motivation
---------	------------

Source cue	■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations (a , d , s) / ■ Consequences
Professor-style explanation	The slide 'Motivation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: ■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations (a , d , s) / ■ Consequences. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Motivation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: ■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations (a , d , s) / ■ Consequences
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Motivation'?
Page 70	Mesh Compression
Source cue	[source]
Professor-style explanation	The slide 'Mesh Compression' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: [source]. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Mesh Compression. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: [source]
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Mesh Compression' into a causal sentence instead of repeating the slide title?
Page 71	Normal Mapping 1/2

Source cue	■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match
Professor-style explanation	The slide 'Normal Mapping 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Normal Mapping 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Mapping 1/2'?
Page 72	Normal Mapping 2/2
Source cue	k_n / ■ During illumination computation d is fetched from the diffuse map, while i is fetched from the / normal map / $i = k \cdot L \cdot k \cdot L \cdot \max(0, n \cdot l) + k \cdot L \cdot \max(0, v \cdot r)$ ■ $p / a + d d s$
Professor-style explanation	The slide 'Normal Mapping 2/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: k_n / ■ During illumination computation d is fetched from the diffuse map, while i is fetched from the / normal map / $i = k \cdot L \cdot k \cdot L \cdot \max(0, n \cdot l) + k \cdot L \cdot \max(0, v \cdot r)$ ■ $p / a + d d s$. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Normal Mapping 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: $k_n \cdot n \cdot L$ / $k_s \cdot \max(0, v \cdot r)$ During illumination computation d is fetched from the diffuse map, while a is fetched from the a normal map / $I = k_d \cdot L \cdot \max(0, n \cdot L) + k_s \cdot L \cdot \max(0, v \cdot r) \cdot p / a + d \cdot s$
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Mapping 2/2'?

Page 73	Normal Maps
Source cue	■ Normal vector components can be positive or negative Components need to be normalized to $[-1, 1]$ / before storage, and rescaled to after texture fetch / $n' := (n) + 0.5$ / Before storage:
Professor-style explanation	The slide 'Normal Maps' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Normal vector components can be positive or negative Components need to be normalized to $[-1, 1]$ / before storage, and rescaled to after texture fetch / $n' := (n) + 0.5$ / Before storage. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Normal Maps. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Normal vector components can be positive or negative Components need to be normalized to $[-1, 1]$ / before storage, and rescaled to after texture fetch / $n' := (n) + 0.5$ / Before storage:
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Maps'?

Page 74	Tangent Space
Source cue	■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by t / s

Professor-style explanation	The slide 'Tangent Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by / Normal $t / t s$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Tangent Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by / Normal $t / t s$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Tangent Space'?
Page 75	TBN Matrix
Source cue	■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent
Professor-style explanation	The slide 'TBN Matrix' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about TBN Matrix. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'TBN Matrix'?
Page 76	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 77	Tangent Vector Derivation 1/2
Source cue	■ ■ / t b / Observation: triangle edges can be expressed based on basis given by and / ■ ■ / e ■ = $\Delta u \cdot t + \Delta v \cdot b$
Professor-style explanation	The slide 'Tangent Vector Derivation 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: ■ ■ / t b / Observation: triangle edges can be expressed based on basis given by and / ■ ■ / e ■ = $\Delta u \cdot t + \Delta v \cdot b$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Tangent Vector Derivation 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} / \begin{bmatrix} t & b \\ 0 & 0 \end{bmatrix} /$ We can solve this matrix for $\begin{bmatrix} t & t & t \\ \Delta u & \Delta v & e \\ e & e & e \end{bmatrix} / \begin{bmatrix} x & y & z \\ 0 & 0 & 0 \end{bmatrix}$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Tangent Vector Derivation 2/2'?

Page 80	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?

Page 81	Observations
Source cue	$\begin{bmatrix} \mathbf{t} & \mathbf{b} & \mathbf{n} \end{bmatrix} / \begin{bmatrix} t & b \\ 0 & 0 \end{bmatrix} /$ and are perpendicular to each other as well as to $\mathbf{t}$ only one, or $\mathbf{n}$, needs to be computed, while the other can be derived through cross product $\mathbf{t} \times \mathbf{b}$
Professor-style explanation	The slide 'Observations' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: $\begin{bmatrix} \mathbf{t} & \mathbf{b} & \mathbf{n} \end{bmatrix} / \begin{bmatrix} t & b \\ 0 & 0 \end{bmatrix} /$ and are perpendicular to each other as well as to $\mathbf{t}$ only one, or $\mathbf{n}$, needs to be computed, while the other can be derived through cross product $\mathbf{t} \times \mathbf{b}$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Observations. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: $\mathbf{n} \cdot \mathbf{b} \Rightarrow \mathbf{t} \cdot \mathbf{b}$ / and are perpendicular to each other as well as to $\mathbf{t}$ only one, or , needs to be / computed, while the other can be derived through cross product / $\mathbf{n} \times \mathbf{b}$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Observations' into a causal sentence instead of repeating the slide title?

Page 82	Variations
Source cue	• Different variations of normal maps exist / • Bump maps - specify displacement of surface points in normal direction / Vector offset bump maps - offset vector added to normal / • Vector rotation bump maps - normal rotated given angle
Professor-style explanation	The slide 'Variations' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: • Different variations of normal maps exist / • Bump maps - specify displacement of surface points in normal direction / Vector offset bump maps - offset vector added to normal / • Vector rotation bump maps - normal rotated given angle. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Variations. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: • Different variations of normal maps exist / • Bump maps - specify displacement of surface points in normal direction / Vector offset bump maps - offset vector added to normal / • Vector rotation bump maps - normal rotated given angle
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Variations'?

Page 83	9.6 Environment Mapping
Source cue	Sampling directional environment data for reflections and lighting

Professor-style explanation	The slide '9.6 Environment Mapping' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Sampling directional environment data for reflections and lighting. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 9.6 Environment Mapping. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Sampling directional environment data for reflections and lighting
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '9.6 Environment Mapping'?

Page 84	Environment Mapping Motivation
Source cue	■ Many reflective and glossy materials show light from their surroundings / ■ Explicitly tracing all reflection rays is expensive in interactive rendering / ■ Environment maps provide a compact representation of incident radiance / ■ Typical use cases • mirror-like reflections
Professor-style explanation	The slide 'Environment Mapping Motivation' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Many reflective and glossy materials show light from their surroundings / ■ Explicitly tracing all reflection rays is expensive in interactive rendering / ■ Environment maps provide a compact representation of incident radiance / ■ Typical use cases / • mirror-like reflections. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Environment Mapping Motivation. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Many reflective and glossy materials show light from their surroundings / ■ Explicitly tracing all reflection rays is expensive in interactive rendering / ■ Environment maps provide a compact representation of incident radiance / ■ Typical use cases / • mirror-like reflections
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.

Check yourself	Can you decide whether 'Environment Mapping Motivation' works per object, per image region, per ray, or per fragment?
Page 85	Environment Mapping Process
Source cue	■ Environment mapping simulates reflections by sampling an image of the surrounding scene ■ The environment is treated as if it were infinitely far away ■ The reflected view direction determines which part of the environment is visible / [Need for Speed no Limits]
Professor-style explanation	The slide 'Environment Mapping Process' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Environment mapping simulates reflections by sampling an image of the surrounding scene ■ The environment is treated as if it were infinitely far away ■ The reflected view direction determines which part of the environment is visible / [Need for Speed no Limits]. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Environment Mapping Process. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Environment mapping simulates reflections by sampling an image of the surrounding scene ■ The environment is treated as if it were infinitely far away ■ The reflected view direction determines which part of the environment is visible / [Need for Speed no Limits]
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Process'?
Page 86	Environment Mapping Assumptions
Source cue	■ The environment is assumed to be far away • translation of the object does not change the sampled environment ■ The reflected object does not reflect itself ■ Nearby dynamic objects are usually missing ■ The approximation works best for

Professor-style explanation	The slide 'Environment Mapping Assumptions' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ The environment is assumed to be far away / • translation of the object does not change the sampled environment / ■ The reflected object does not reflect itself / ■ Nearby dynamic objects are usually missing / ■ The approximation works best for. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Environment Mapping Assumptions. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ The environment is assumed to be far away / • translation of the object does not change the sampled environment / ■ The reflected object does not reflect itself / ■ Nearby dynamic objects are usually missing / ■ The approximation works best for
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Assumptions'?
Page 87	Environment Mapping 2/2
Source cue	$V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model / $R = 2 \cdot (V \cdot N) \cdot N - V$
Professor-style explanation	The slide 'Environment Mapping 2/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: $V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model / $R = 2 \cdot (V \cdot N) \cdot N - V$. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Environment Mapping 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: $V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model / $R = 2 \cdot (V \cdot N) \cdot N - V$
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Environment Mapping 2/2'?

Page 88	Demo - Cube-map Reflection Vector
Source cue	Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1
Professor-style explanation	The slide 'Demo - Cube-map Reflection Vector' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Demo - Cube-map Reflection Vector. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Demo - Cube-map Reflection Vector' into a causal sentence instead of repeating the slide title?

Page 89	Environment Mapping Types 1/2
Source cue	■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support

Professor-style explanation	The slide 'Environment Mapping Types 1/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Environment Mapping Types 1/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Types 1/2'?

Page 90	Environment Mapping Types 2/2
Source cue	■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps
Professor-style explanation	The slide 'Environment Mapping Types 2/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Environment Mapping Types 2/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps

Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Types 2/2'?
Page 91	Spherical Mapping 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Spherical Mapping 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Spherical Mapping 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Spherical Mapping 1/2' into a causal sentence instead of repeating the slide title?
Page 92	Spherical Mapping 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Spherical Mapping 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Spherical Mapping 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Spherical Mapping 2/2' into a causal sentence instead of repeating the slide title?
Page 93	Spherical Environment Mapping

Source cue	■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion
Professor-style explanation	The slide 'Spherical Environment Mapping' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Spherical Environment Mapping. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping'?
Page 94	Sphere Map Distortion
Source cue	■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred

Professor-style explanation	The slide 'Sphere Map Distortion' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Sphere Map Distortion. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Sphere Map Distortion'?
Page 95	Demo - Environment Map Distortion
Source cue	Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection
Professor-style explanation	The slide 'Demo - Environment Map Distortion' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Demo - Environment Map Distortion. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection

Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Environment Map Distortion'?
Page 96	Sphere Map Interpolation
Source cue	■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted
Professor-style explanation	The slide 'Sphere Map Interpolation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Sphere Map Interpolation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Sphere Map Interpolation' into a causal sentence instead of repeating the slide title?
Page 97	Sphere Map Coordinates
Source cue	■ Sphere map texture coordinates are calculated in camera coordinates / (0, 0, 0) / • Camera in origin / V -z +x +y / • View direction along the -axis (axis to the right, axis up)

Professor-style explanation	The slide 'Sphere Map Coordinates' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Sphere map texture coordinates are calculated in camera coordinates / $(0, 0, 0) / \bullet$ Camera in origin / $V -z +x +y / \bullet$ View direction along the -axis (axis to the right, axis up). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Sphere Map Coordinates. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Sphere map texture coordinates are calculated in camera coordinates / $(0, 0, 0) / \bullet$ Camera in origin / $V -z +x +y / \bullet$ View direction along the -axis (axis to the right, axis up)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Sphere Map Coordinates'?

Page 98	Cube Environment Mapping
Source cue	■ Cube maps are represented by 6 square textures forming the sides of a cube / ■ The cube is centered around the shading point or camera / ■ Each face stores the environment as seen through one side of the cube / ■ A 3D direction vector is used to select the face and lookup position
Professor-style explanation	The slide 'Cube Environment Mapping' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: ■ Cube maps are represented by 6 square textures forming the sides of a cube / ■ The cube is centered around the shading point or camera / ■ Each face stores the environment as seen through one side of the cube / ■ A 3D direction vector is used to select the face and lookup position. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Cube Environment Mapping. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: ■ Cube maps are represented by 6 square textures forming the sides of a cube / ■ The cube is centered around the shading point or camera / ■ Each face stores the environment as seen through one side of the cube / ■ A 3D direction vector is used to select the face and lookup position
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Cube Environment Mapping' changes positions before rasterization?

Page 99	Cube Mapping Example 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Cube Mapping Example 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Cube Mapping Example 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Cube Mapping Example 1/2' into a causal sentence instead of repeating the slide title?

Page 100	Cube Mapping Example 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Cube Mapping Example 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Cube Mapping Example 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Cube Mapping Example 2/2' into a causal sentence instead of repeating the slide title?
Page 101	Cube Mapping Distortion
Source cue	■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries
Professor-style explanation	The slide 'Cube Mapping Distortion' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Cube Mapping Distortion. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Cube Mapping Distortion'?
Page 102	Cube Map Texture Coordinates
Source cue	$R = (R_x, R_y, R_z)$ / ■ The reflection vector $x\ y\ z$ is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of R / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face

Professor-style explanation	The slide 'Cube Map Texture Coordinates' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $R = (R_x, R_y, R_z)$ / ■ The reflection vector x, y, z is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Cube Map Texture Coordinates. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $R = (R_x, R_y, R_z)$ / ■ The reflection vector x, y, z is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Cube Map Texture Coordinates'?
Page 103	Environment Mapping in OpenGL
Source cue	■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects
Professor-style explanation	The slide 'Environment Mapping in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Environment Mapping in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Environment Mapping in OpenGL'?

Page 104	Cube Map Texture Object
Source cue	■ Cube maps are OpenGL texture objects with target GL_TEXTURE_CUBE_MAP / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 GLuint envMap; / 2 glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &envMap);
Professor-style explanation	The slide 'Cube Map Texture Object' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Cube maps are OpenGL texture objects with target GL_TEXTURE_CUBE_MAP / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 GLuint envMap; / 2 glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &envMap). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Cube Map Texture Object. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Cube maps are OpenGL texture objects with target GL_TEXTURE_CUBE_MAP / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 GLuint envMap; / 2 glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &envMap);
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Cube Map Texture Object'?
Page 105	Sampling Direction

Source cue	■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition);
Professor-style explanation	The slide 'Sampling Direction' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: ■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Sampling Direction. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: ■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition);
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Sampling Direction' changes positions before rasterization?
Page 106	Environment Mapping in GLSL 1/2
Source cue	■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader

Professor-style explanation	The slide 'Environment Mapping in GLSL 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Environment Mapping in GLSL 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Environment Mapping in GLSL 1/2'?
Page 107	Environment Mapping in GLSL 2/2
Source cue	1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N);
Professor-style explanation	The slide 'Environment Mapping in GLSL 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: 1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Environment Mapping in GLSL 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: 1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N);
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Environment Mapping in GLSL 2/2' changes positions before rasterization?
Page 108	Practical Cube-Map Issues
Source cue	■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details
Professor-style explanation	The slide 'Practical Cube-Map Issues' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Practical Cube-Map Issues. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Practical Cube-Map Issues'?
Page 109	1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);
Source cue	3 glTextureParameteri(envMap, / Glass Utah teapot rendering; image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]

Professor-style explanation	The slide '1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: 3 glTextureParameteri(envMap, / Glass Utah teapot rendering; image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about 1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: 3 glTextureParameteri(envMap, / Glass Utah teapot rendering; image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);'?
Page 110	Cube Mapping Pros and Cons
Source cue	■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data
Professor-style explanation	The slide 'Cube Mapping Pros and Cons' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Cube Mapping Pros and Cons. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Cube Mapping Pros and Cons' into a causal sentence instead of repeating the slide title?
Page 111	Spherical Environment Mapping Assumptions

Source cue	■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account
Professor-style explanation	The slide 'Spherical Environment Mapping Assumptions' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Spherical Environment Mapping Assumptions. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping Assumptions'?
Page 112	Spherical Environment Mapping Conclusions
Source cue	■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border
Professor-style explanation	The slide 'Spherical Environment Mapping Conclusions' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.

Technical commentary	This slide is about Spherical Environment Mapping Conclusions. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping Conclusions'?
Page 113	9.7 3D Textures and Volume Rendering
Source cue	Texturing volumetric data, solid materials, and participating media
Professor-style explanation	The slide '9.7 3D Textures and Volume Rendering' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Texturing volumetric data, solid materials, and participating media. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 9.7 3D Textures and Volume Rendering. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Texturing volumetric data, solid materials, and participating media
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '9.7 3D Textures and Volume Rendering'?
Page 114	3D Texturing: Relevant Topics
Source cue	■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations

Professor-style explanation	The slide '3D Texturing: Relevant Topics' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3D Texturing: Relevant Topics. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3D Texturing: Relevant Topics'?
Page 115	3D Texture as a Voxel Grid
Source cue	■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity
Professor-style explanation	The slide '3D Texture as a Voxel Grid' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about 3D Texture as a Voxel Grid. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '3D Texture as a Voxel Grid'?

Page 116	Object Space to Texture Space
Source cue	■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$
Professor-style explanation	The slide 'Object Space to Texture Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Object Space to Texture Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Object Space to Texture Space'?

Page 117	3D Texture Access
Source cue	■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation

Professor-style explanation	The slide '3D Texture Access' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3D Texture Access. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3D Texture Access'?
Page 118	Trilinear Interpolation
Source cue	■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along
Professor-style explanation	The slide 'Trilinear Interpolation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Trilinear Interpolation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Trilinear Interpolation' into a causal sentence instead of repeating the slide title?

Page 119	$f = (1 - \alpha) \cdot f + \alpha \cdot f$
Source cue	00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110
Professor-style explanation	The slide ' $f = (1 - \alpha) \cdot f + \alpha \cdot f$ ' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: 00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about $f = (1 - \alpha) \cdot f + \alpha \cdot f$. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: 00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn ' $f = (1 - \alpha) \cdot f + \alpha \cdot f$ ' into a causal sentence instead of repeating the slide title?
Page 120	3D Texture Memory Footprint
Source cue	■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB
Professor-style explanation	The slide '3D Texture Memory Footprint' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about 3D Texture Memory Footprint. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '3D Texture Memory Footprint'?

Page 121	3D Mipmaps and Filtering
Source cue	■ 3D textures can use mipmaps analogously to 2D textures / ■ Each mip level halves width, height, and depth / $k + 1$ / ■ Level contains one eighth of the voxels of level / ■ Useful for
Professor-style explanation	The slide '3D Mipmaps and Filtering' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ 3D textures can use mipmaps analogously to 2D textures / ■ Each mip level halves width, height, and depth / $k + 1$ / ■ Level contains one eighth of the voxels of level / ■ Useful for. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about 3D Mipmaps and Filtering. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ 3D textures can use mipmaps analogously to 2D textures / ■ Each mip level halves width, height, and depth / $k + 1$ / ■ Level contains one eighth of the voxels of level / ■ Useful for
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '3D Mipmaps and Filtering' works per object, per image region, per ray, or per fragment?
Page 122	Solid Texturing
Source cue	■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain
Professor-style explanation	The slide 'Solid Texturing' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Solid Texturing. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Solid Texturing'?

Page 123	Volumetric Data as Texture Data
Source cue	■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density
Professor-style explanation	The slide 'Volumetric Data as Texture Data' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Volumetric Data as Texture Data. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Volumetric Data as Texture Data'?

Page 124	Transfer Functions
Source cue	■ A transfer function maps data values to optical properties / $T(\rho) = (c, \alpha)$ / ■ Color emphasizes material classes or value ranges / ■ Opacity controls which structures become visible / ■ Common choices

Professor-style explanation	The slide 'Transfer Functions' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ A transfer function maps data values to optical properties / $T(p) = (c, \alpha)$ / ■ Color emphasizes material classes or value ranges / ■ Opacity controls which structures become visible / ■ Common choices. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Transfer Functions. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ A transfer function maps data values to optical properties / $T(p) = (c, \alpha)$ / ■ Color emphasizes material classes or value ranges / ■ Opacity controls which structures become visible / ■ Common choices
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Transfer Functions'?

Page 125	Volume Rendering with 3D Textures
Source cue	■ 3D grid data is stored in a 3D texture / ■ Slice-based volume rendering draws proxy geometry through the volume / ■ Slices are usually orthogonal to the view direction / ■ Each slice samples the 3D texture and applies the transfer function / ■ Slices are blended back to front or front to back
Professor-style explanation	The slide 'Volume Rendering with 3D Textures' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ 3D grid data is stored in a 3D texture / ■ Slice-based volume rendering draws proxy geometry through the volume / ■ Slices are usually orthogonal to the view direction / ■ Each slice samples the 3D texture and applies the transfer function / ■ Slices are blended back to front or front to back. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.

Technical commentary	This slide is about Volume Rendering with 3D Textures. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ 3D grid data is stored in a 3D texture ■ Slice-based volume rendering draws proxy geometry through the volume ■ Slices are usually orthogonal to the view direction ■ Each slice samples the 3D texture and applies the transfer function ■ Slices are blended back to front or front to back
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Volume Rendering with 3D Textures'?

Page 126	Front-to-Back Alpha Compositing
Source cue	■ Volume rendering approximates absorption and emission along a ray / $c \cdot \alpha$ ■ For a sample with color i and opacity α / $C = C + (1 - \alpha) \cdot \alpha \cdot c$ / out in in i
Professor-style explanation	The slide 'Front-to-Back Alpha Compositing' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Volume rendering approximates absorption and emission along a ray / $c \cdot \alpha$ ■ For a sample with color i and opacity α / $C = C + (1 - \alpha) \cdot \alpha \cdot c$ / out in in i The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Front-to-Back Alpha Compositing. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Volume rendering approximates absorption and emission along a ray / $c \cdot \alpha$ ■ For a sample with color i and opacity α / $C = C + (1 - \alpha) \cdot \alpha \cdot c$ / out in in i
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Front-to-Back Alpha Compositing' works per object, per image region, per ray, or per fragment?

Page 127	Ray Marching Volume Rendering
Source cue	■ Instead of drawing slices, cast one ray per fragment through the volume ■ Repeatedly sample the 3D texture along the ray ■ Apply the transfer function at every sample ■ Composite samples until the ray exits the volume or becomes opaque / 1 for (float $t = t_{\text{Entry}}$; $t < t_{\text{Exit}}$; $t += \text{stepSize}$) {

Professor-style explanation	The slide 'Ray Marching Volume Rendering' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Instead of drawing slices, cast one ray per fragment through the volume ■ Repeatedly sample the 3D texture along the ray ■ Apply the transfer function at every sample ■ Composite samples until the ray exits the volume or becomes opaque 1 for (float t = tEntry; t < tExit; t += stepSize) { . The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Ray Marching Volume Rendering. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Instead of drawing slices, cast one ray per fragment through the volume ■ Repeatedly sample the 3D texture along the ray ■ Apply the transfer function at every sample ■ Composite samples until the ray exits the volume or becomes opaque 1 for (float t = tEntry; t < tExit; t += stepSize) {
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Ray Marching Volume Rendering' works per object, per image region, per ray, or per fragment?
Page 128	Gradients and Volume Lighting
Source cue	■ Gradients estimate local orientation in the scalar field ■ Central differences approximate the gradient $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z) / \nabla \rho \approx \rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$
Professor-style explanation	The slide 'Gradients and Volume Lighting' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Gradients estimate local orientation in the scalar field ■ Central differences approximate the gradient $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z) / \nabla \rho \approx \rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$ ■. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Gradients and Volume Lighting. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Gradients estimate local orientation in the scalar field / ■ Central differences approximate the gradient / $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z)$ / ■ ■ / $\nabla \rho \approx$ ■ $\rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$ ■
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Gradients and Volume Lighting'?
Page 129	Quality and Performance Trade-Offs
Source cue	■ Step size / • smaller steps improve quality but increase cost / • larger steps are faster but can miss thin structures / ■ Opacity correction / • keeps appearance stable when step size changes
Professor-style explanation	The slide 'Quality and Performance Trade-Offs' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Step size / • smaller steps improve quality but increase cost / • larger steps are faster but can miss thin structures / ■ Opacity correction / • keeps appearance stable when step size changes. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Quality and Performance Trade-Offs. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Step size / • smaller steps improve quality but increase cost / • larger steps are faster but can miss thin structures / ■ Opacity correction / • keeps appearance stable when step size changes
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Quality and Performance Trade-Offs' into a causal sentence instead of repeating the slide title?
Page 130	3D Texturing Takeaways
Source cue	■ 3D textures generalize 2D texture lookup from surfaces to volumes / ■ They are useful for solid materials, volumetric effects, and sampled scientific data / ■ Trilinear interpolation provides smooth access to voxel grids / ■ Volume rendering turns sampled data into color and opacity along viewing rays / ■ Transfer functions and compositing determine what becomes visible

Professor-style explanation	The slide '3D Texturing Takeaways' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ 3D textures generalize 2D texture lookup from surfaces to volumes ■ They are useful for solid materials, volumetric effects, and sampled scientific data ■ Trilinear interpolation provides smooth access to voxel grids ■ Volume rendering turns sampled data into color and opacity along viewing rays ■ Transfer functions and compositing determine what becomes visible. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about 3D Texturing Takeaways. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ 3D textures generalize 2D texture lookup from surfaces to volumes ■ They are useful for solid materials, volumetric effects, and sampled scientific data ■ Trilinear interpolation provides smooth access to voxel grids ■ Volume rendering turns sampled data into color and opacity along viewing rays ■ Transfer functions and compositing determine what becomes visible
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '3D Texturing Takeaways' works per object, per image region, per ray, or per fragment?
Page 131	■ Textures are sampled GPU data sources used by shaders
Source cue	• Color, opacity, normals, roughness, height, masks, and volume density ■ Texture coordinates define where texture data is sampled / (s, t, r) • UV mapping, texture transformations, and 3D coordinates ■ Texture formats, channel semantics, and color space determine correct interpretation

Professor-style explanation	The slide '■ Textures are sampled GPU data sources used by shaders' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: • Color, opacity, normals, roughness, height, masks, and volume density / ■ Texture coordinates define where texture data is sampled / (s, t, r) / • UV mapping, texture transformations, and 3D coordinates / ■ Texture formats, channel semantics, and color space determine correct interpretation. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about ■ Textures are sampled GPU data sources used by shaders. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: • Color, opacity, normals, roughness, height, masks, and volume density / ■ Texture coordinates define where texture data is sampled / (s, t, r) / • UV mapping, texture transformations, and 3D coordinates / ■ Texture formats, channel semantics, and color space determine correct interpretation
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '■ Textures are sampled GPU data sources used by shaders'?

Page 132	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Literature and other sources used in this chapter' into a causal sentence instead of repeating the slide title?
Page 133	■ Text Books
Source cue	• J. Foley, A. van Dam, S. Fine"Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others
Professor-style explanation	The slide '■ Text Books' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: • J. Foley, A. van Dam, S. Fine"Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about ■ Text Books. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: • J. Foley, A. van Dam, S. Fine"Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '■ Text Books' into a causal sentence instead of repeating the slide title?

09.1 Texture Objects, Texels, Formats, and Color Space

Source location: Section 9.1

Commentary

A texture object stores sampled data on the GPU. A texel is one stored texture sample. The texture format decides what channels and numeric representation the samples use. Color space matters because color values may be stored nonlinearly, especially for sRGB images.

The key shift is to stop thinking of texture as only a picture. Textures can store color, normals, depth, lookup data, environment information, or volume data. The shader decides how the sampled values are interpreted.

Mental model

A texture is a GPU-accessible sampled data field.

Check yourself

Why can the same texture mechanism store both color maps and normal maps?

09.2 Texture Coordinates, UV Mapping, and Wrapping

Source location: Section 9.2

Commentary

Texture coordinates map surface points to locations in texture space. UV coordinates are usually interpolated across primitives during rasterization and then used by the fragment shader to sample a texture.

Wrapping rules decide what happens outside the normal coordinate range. Repeat, clamp, mirrored repeat, and border behavior are not visual afterthoughts; they define the sampling function outside the base domain.

Mental model

UVs are the address system that lets a fragment look up texture data.

Check yourself

What artifact might appear if a wrapping mode is wrong at the edge of a surface?

09.3 Sampling, Filtering, Mipmaps, and Anisotropy

Source location: Section 9.3

Commentary

Sampling turns continuous texture coordinates into values from discrete texels. Nearest filtering selects a nearby texel and can look blocky. Linear filtering blends nearby texels and looks smoother. Minification is harder because many texels may map to one pixel.

Mipmaps store prefiltered lower-resolution versions of a texture. They reduce aliasing and shimmer when textures are seen at small scale. Anisotropic filtering improves quality when a texture is viewed at a steep angle where footprint shape is elongated.

Mental model

Filtering reconstructs values; mipmaps choose an appropriate scale before reconstruction.

Check yourself

Why does a distant checkerboard shimmer without mipmapping?

09.4 Modern OpenGL Texture Pipeline

Source location: Section 9.4

Commentary

In modern OpenGL, textures are represented by texture objects, bound to texture units, connected to sampler uniforms, and accessed in shaders. The shader does not sample a filename; it samples a bound texture through a sampler.

Texture bugs often come from state mismatches: wrong active texture unit, wrong sampler uniform, missing mipmaps for a mipmap filter, wrong wrap mode, or wrong internal format.

Mental model

OpenGL texturing is a chain: object data -> texture unit -> sampler uniform -> shader lookup.

Check yourself

Why can a texture appear black when the shader code is mathematically correct?

09.5 Normal Mapping

Source location: Section 9.5

Commentary

Normal mapping stores normal directions in a texture so that lighting can vary at a finer scale than the geometry. The surface may have few triangles, but the shader receives detailed normals per fragment.

The hard part is coordinate space. Normal maps are often defined in tangent space, so the shader must transform or interpret them with the correct tangent, bitangent, and normal basis.

Mental model

Normal mapping changes the lighting normal, not the actual mesh silhouette.

Check yourself

Why does normal mapping not change the geometric outline of an object?

09.6 Environment Mapping and 3D Textures

Source location: Sections 9.6-9.7

Commentary

Environment mapping uses textures to represent surrounding illumination or reflections. A direction, not a surface UV alone, can be used to sample an environment map. This is a texture lookup driven by view/reflection geometry.

3D textures and volume rendering extend the same sampled-data idea into three dimensions. Instead of sampling a 2D image, the shader samples a volume. This is useful for data such as medical scans, density fields, or procedural volumetric effects.

Mental model

Textures are general sampled data; the coordinate dimensionality depends on the problem.

Check yourself

What coordinate type would you use to sample a 3D texture?

End-of-lecture summary

If you remember only one thing from Lecture 09, remember this: This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.